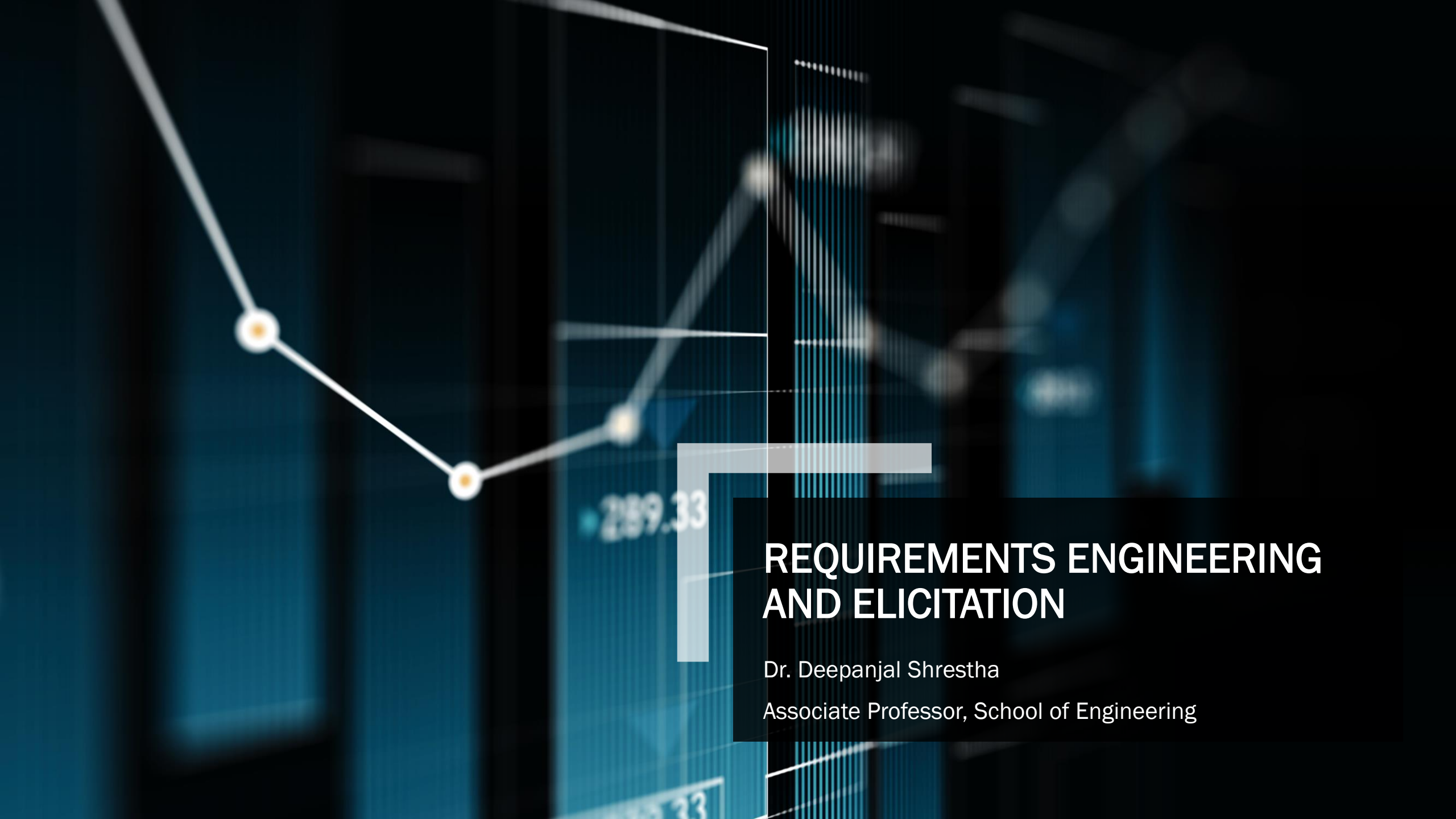


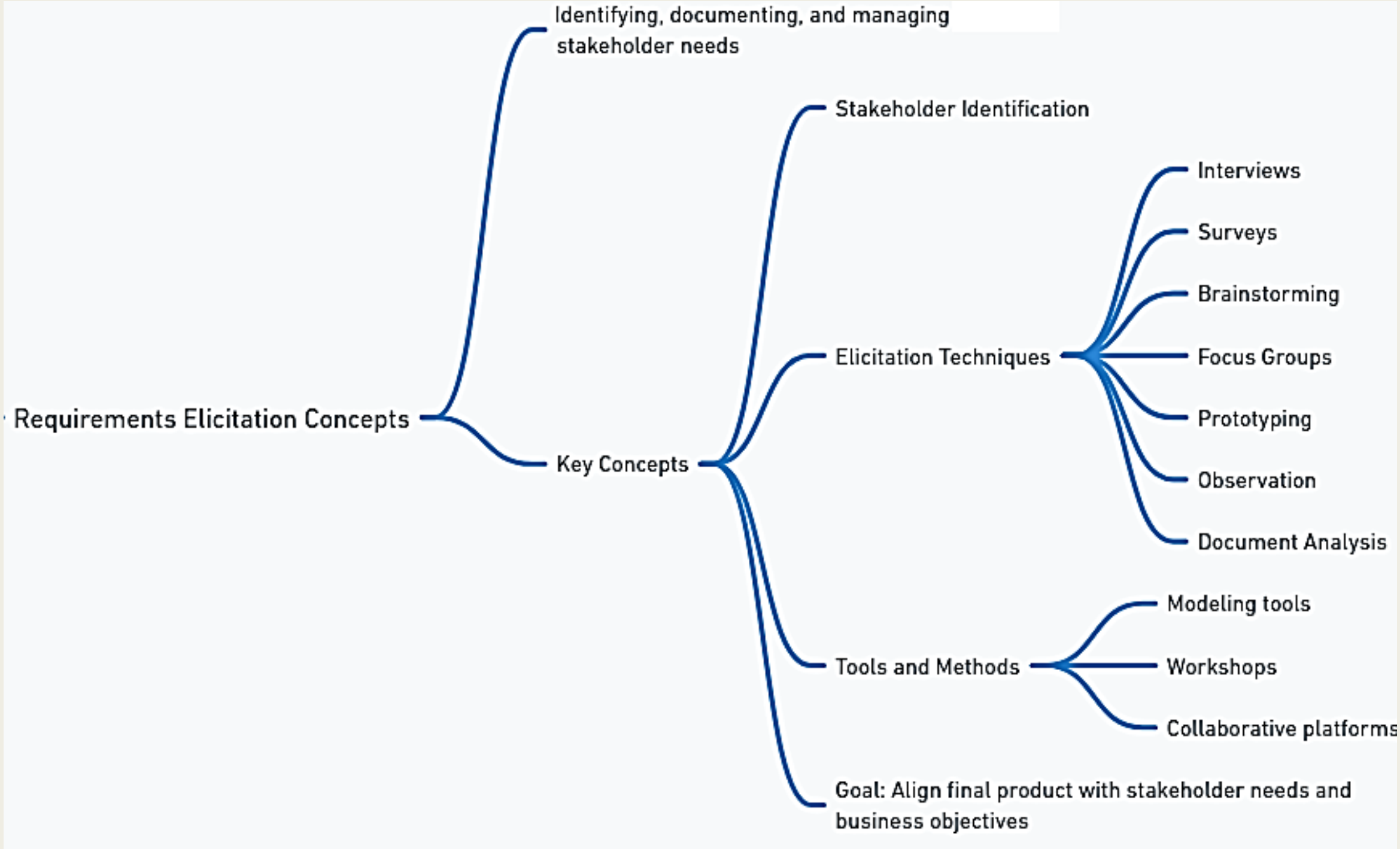
REQUIREMENTS ENGINEERING AND ELICITATION

Dr. Deepanjal Shrestha

Associate Professor, School of Engineering

Introduction

- **What is Requirements Elicitation?**
  - The process of **discovering, documenting**, and **validating** the requirements of a system.
  - Involves **understanding the needs of stakeholders** and **translating them into specific, measurable requirements**.
- **Why is it Important?**
  - **Directly impacts project success**: A well-defined requirements document significantly reduces the risk of project failure.
  - **Cost-effective**: Identifying and addressing issues early in the development cycle saves time and money.
  - **Customer Satisfaction**: Ensures that the final product meets the needs and expectations of users.


Requirements Elicitation Concepts
Identifying, documenting, and managing stakeholder needs
Key Concepts
Stakeholder Identification
Elicitation Techniques
Interviews
Surveys
Brainstorming
Focus Groups
Prototyping
Observation
Document Analysis
Tools and Methods
Modeling tools
Workshops
Collaborative platforms
Goal: Align final product with stakeholder needs and business objectives

The Importance of Stakeholder Involvement

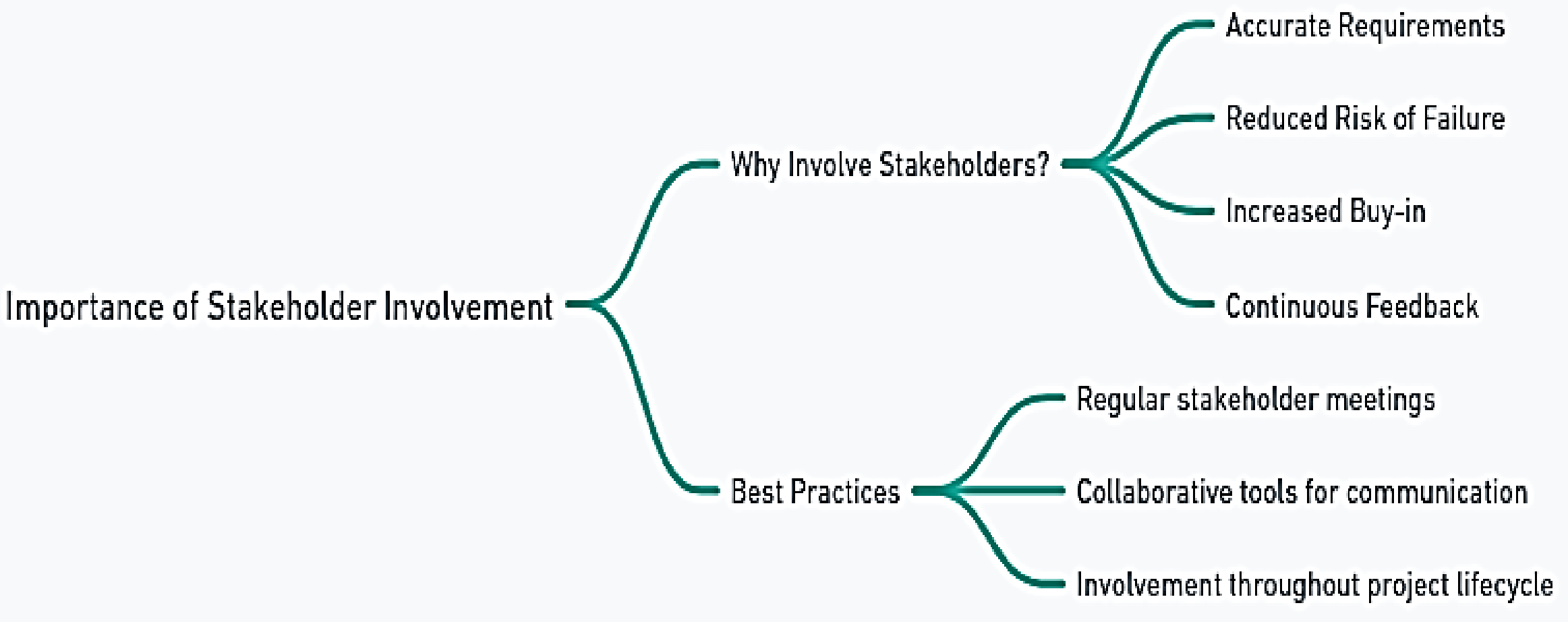
- **Who are Stakeholders?**
  - **End-users:** Those who directly interact with the system.
  - **Domain experts**: Individuals with deep knowledge of the business domain.
  - **Project managers:** Oversee the project's timeline and budget.
  - **Developers:** Responsible for designing and implementing the system.
  - **Customers:** The individuals or organizations purchasing the system.

Benefits of Stakeholder Involvement

- **Enhanced Requirements Quality**: Direct input from stakeholders leads to more accurate and complete requirements.
- **Increased User Satisfaction:** By involving users early on, their needs and preferences are better understood.
- **Reduced Risk of Project Failure:** Early identification and resolution of issues.
- **Improved Communication and Collaboration:** Fosters a collaborative environment and reduces misunderstandings.

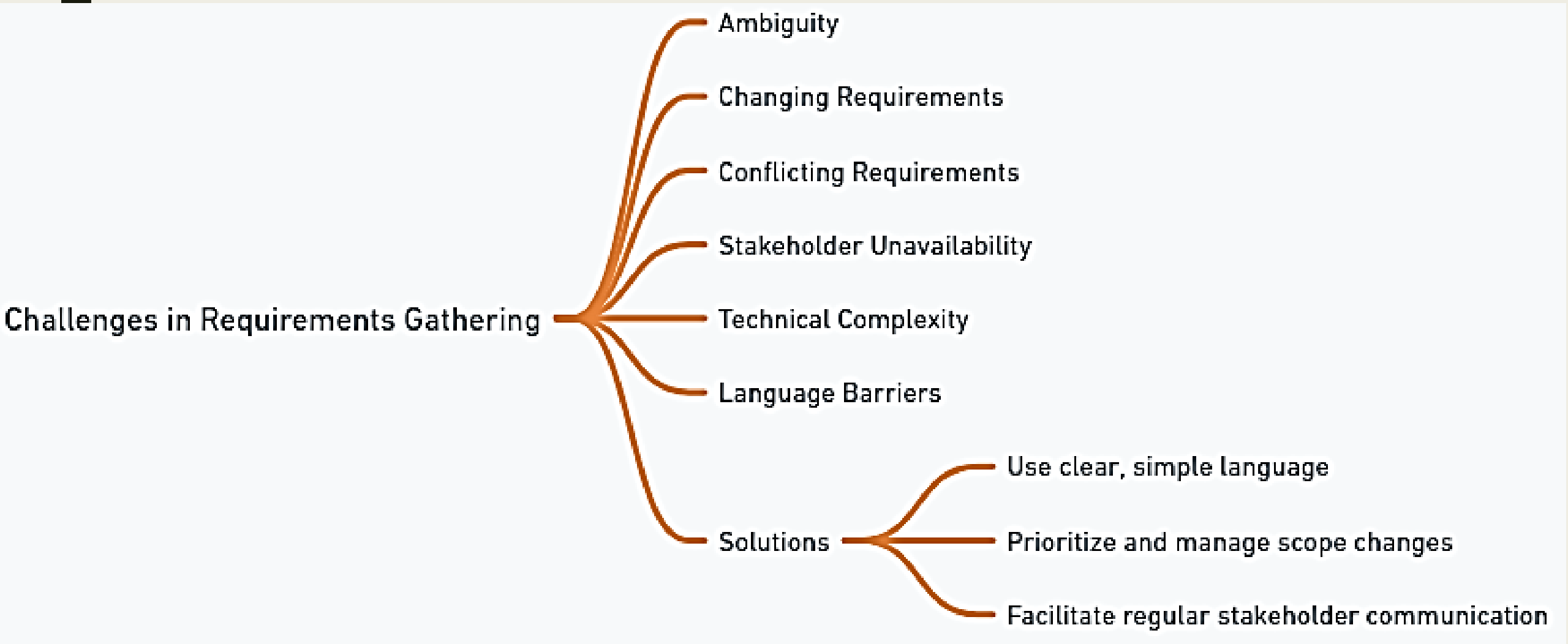


Case Study: The failed launch of Microsoft's Windows Vista was largely attributed to poor requirements gathering and a lack of stakeholder involvement.


Importance of Stakeholder Involvement
Why Involve Stakeholders?
Accurate Requirements
Reduced Risk of Failure
Increased Buy-in
Continuous Feedback
Best Practices
Regular stakeholder meetings
Collaborative tools for communication
Involvement throughout project lifecycle

Challenges in Requirements Gathering

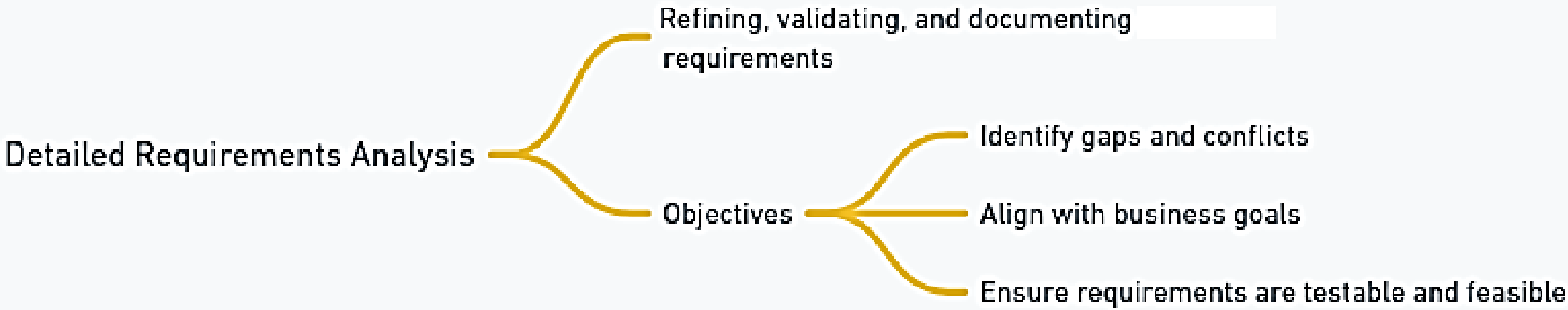
- **Communication Barriers**: Misunderstandings, language differences, and cultural differences can hinder effective communication.
- **Evolving Requirements**: Business needs and technological advancements can lead to changing requirements.
- **Conflicting Priorities**: Different stakeholders may have competing interests.
- **Incomplete or Ambiguous Requirements:** Vague or unclear requirements can lead to misinterpretations.
- **Lack of Stakeholder Involvement:** Insufficient input from key stakeholders can result in systems that do not meet their needs.


Challenges in Requirements Gathering
Ambiguity
Changing Requirements
Conflicting Requirements
Stakeholder Unavailability
Technical Complexity
Language Barriers
Solutions
Use clear, simple language
Prioritize and manage scope changes
Facilitate regular stakeholder communication

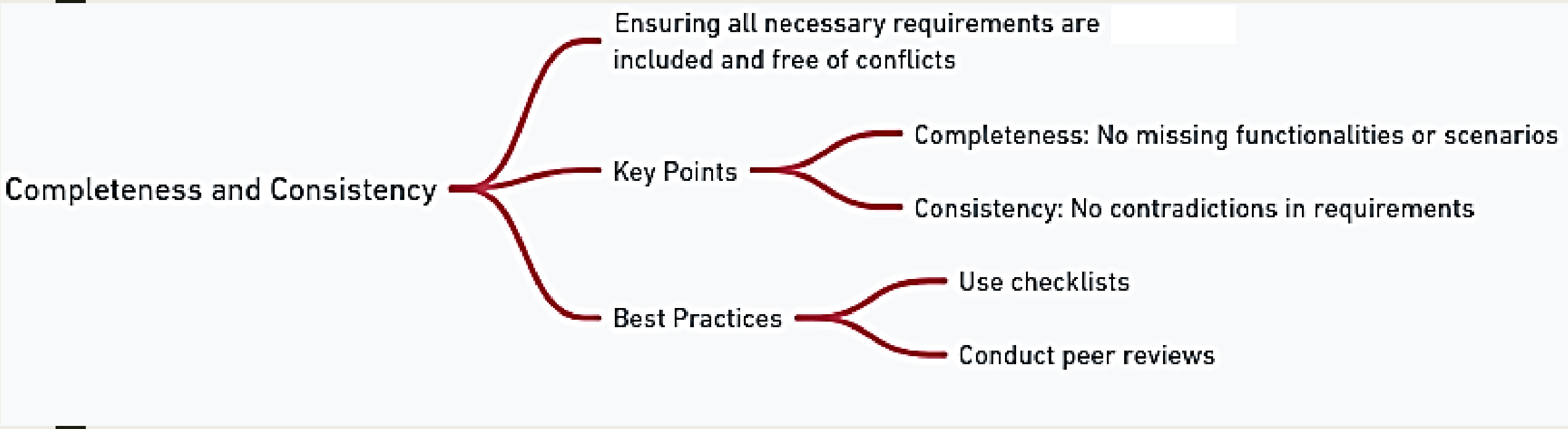
Detailed Requirements Analysis

- To ensure the quality of requirements, consider the following attributes:
  - Completeness: All necessary requirements are identified.
  - Consistency: Requirements are not contradictory or conflicting.
  - Clarity and Correctness: Requirements are clear, unambiguous, and accurate.
  - Realism and Verifiability: Requirements are feasible and can be verified through testing.
  - Traceability: Requirements can be traced back to their source (e.g., user stories, use cases).

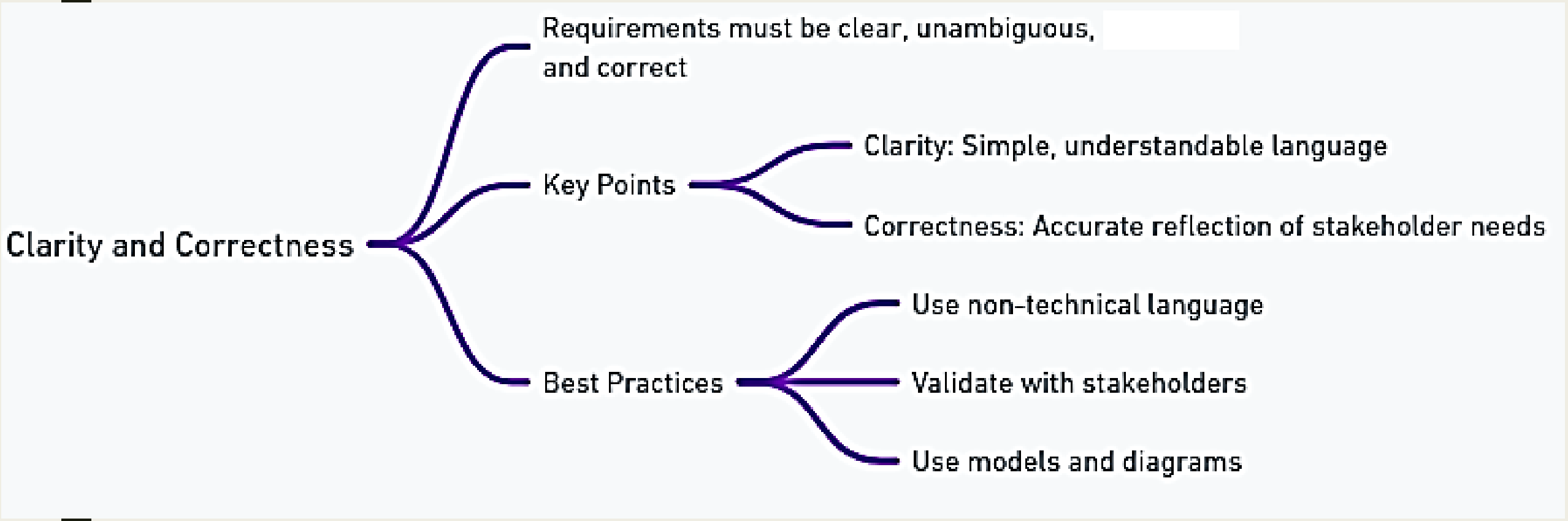
Detailed Requirements Analysis



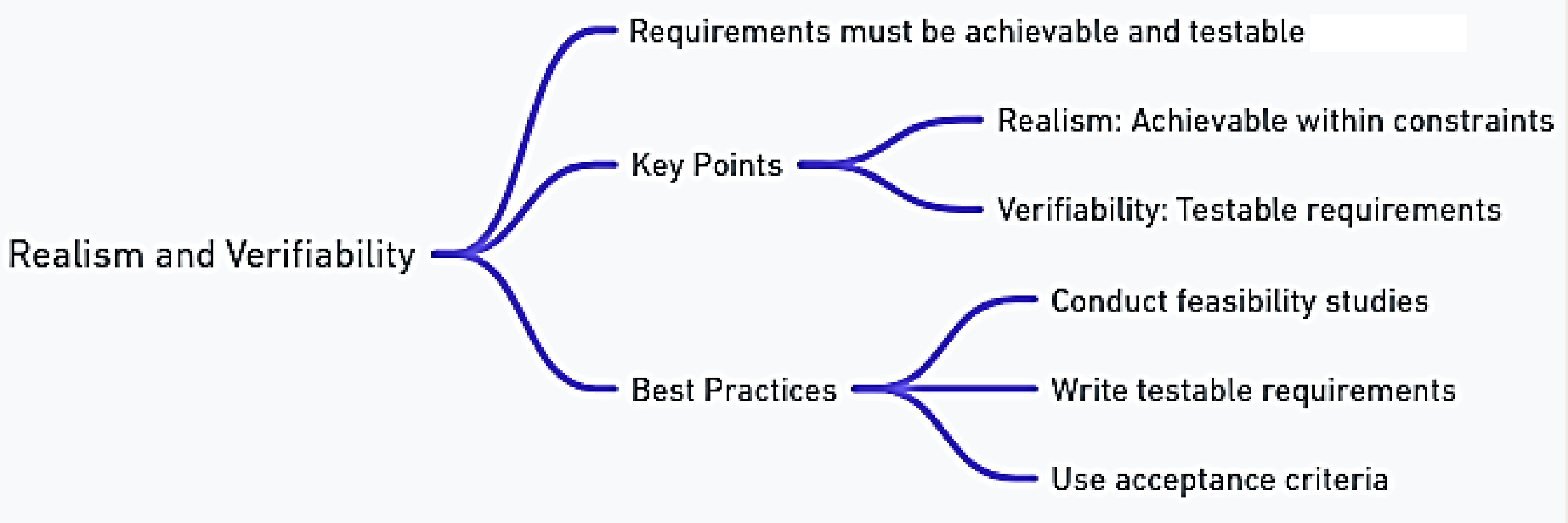
Attributes of a Requirements Analysis



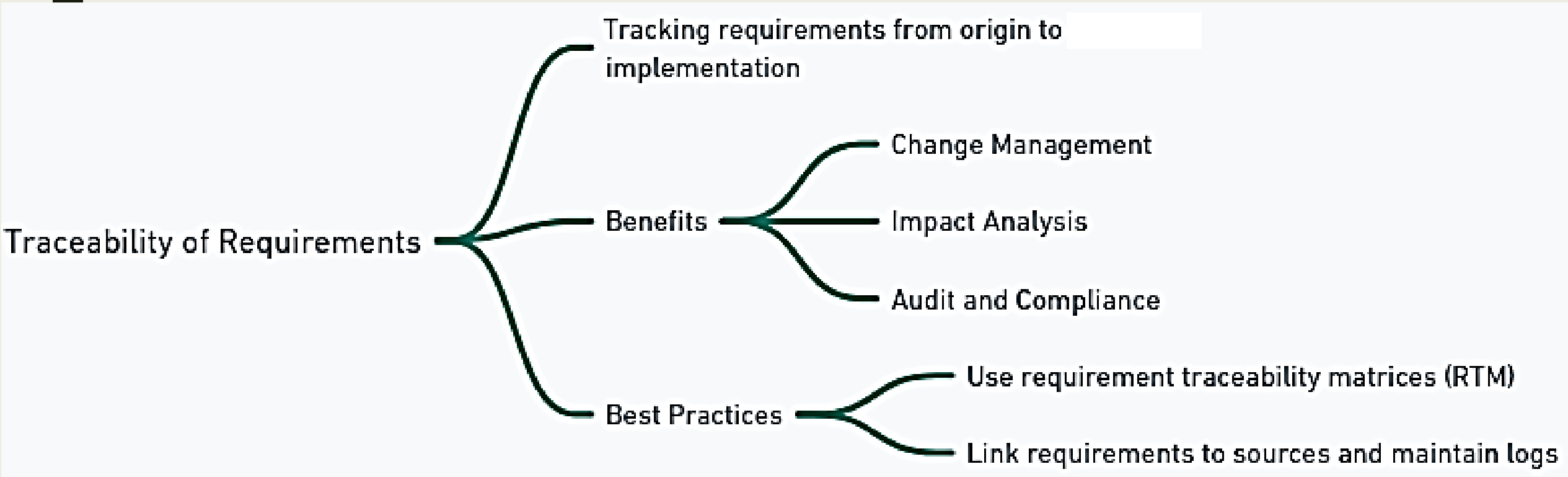
Attributes of a Requirements Analysis



Attributes of a Requirements Analysis



Attributes of a Requirements Analysis



REQUIREMENTS TRACEABILITY MATRIX

Project Name: Online Flight Booking Application

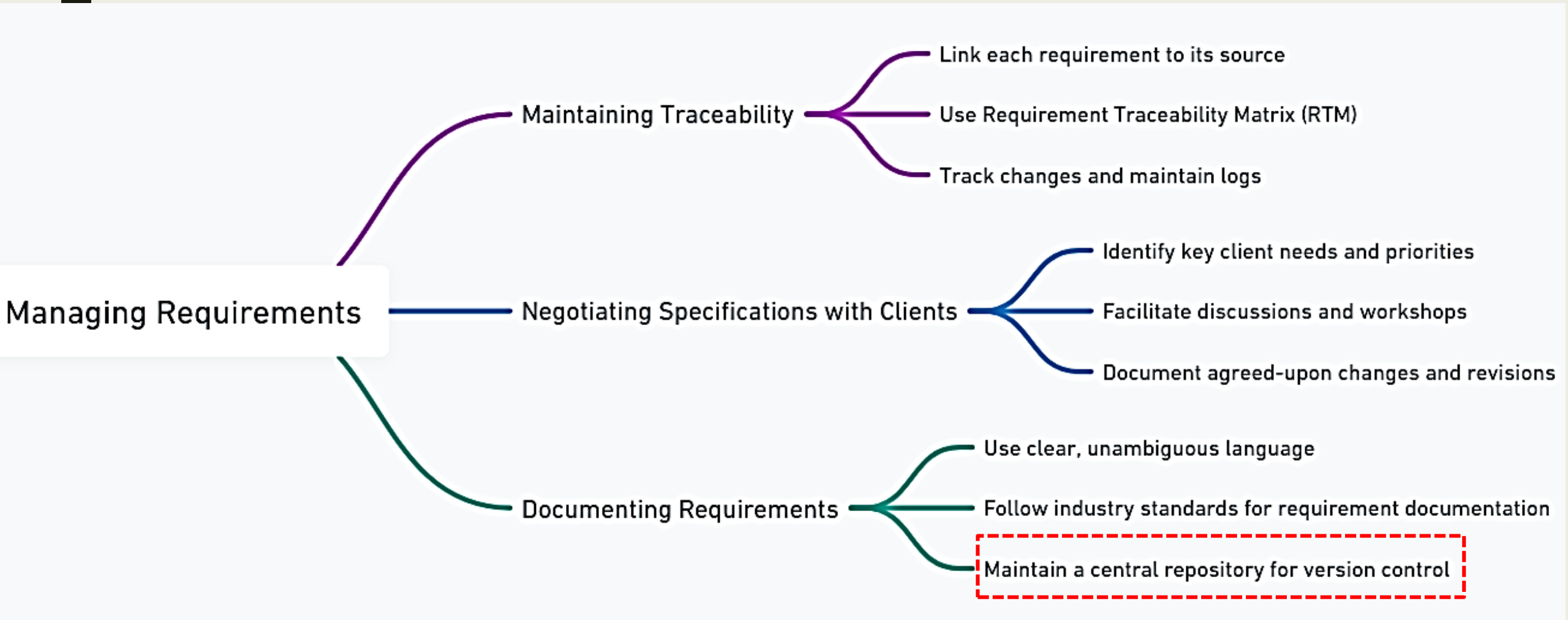
Business Requirements Document BRD		Functional Requirements Document FSD			Test Case Document
Business Requirement ID#	Business Requirement / Business Use case	Functional Requirement ID#	Functional Requirement / Use Case	Priority	Test Case ID#
BR_1	Reservation Module	FR_1	One Way Ticket booking	High	TC#001 TC#002
		FR_2	Round Way Ticket		TC#003 TC#004
		FR_3	Multicity Ticket booking	High	TC#005 TC#006
BR_2	Payment Module	FR_4	By Credit Card	High	TC#007 TC#008
		FR_5	By Debit Card	High	TC#009
		FR_6	By Reward Points	Medium	TC#010 TC#011

Requirement Elicitation Techniques

- **Interviews:** One-on-one conversations with stakeholders to gather information.
- **Surveys and Questionnaires:** Collect quantitative data from a large number of stakeholders.
- **Workshops:** Facilitate collaborative discussions among stakeholders to identify requirements.
- **Focus Groups:** Conduct group discussions with specific stakeholder groups.
- **Observation:** Observe users interacting with existing systems or prototypes.
- **Document Analysis:** Review existing documentation, such as business process models and system specifications.

Requirements Elicitation Techniques

- **Use Case Diagrams:** Visualize the interactions between actors and the system.
- **User Stories:** Describe user goals and acceptance criteria in a simple format (e.g., As a [user role], I want to [goal] so that [reason]).
- **Process Models**: Model business processes to identify system requirements.
- **Prototyping:** Create early prototypes to gather feedback and refine requirements.

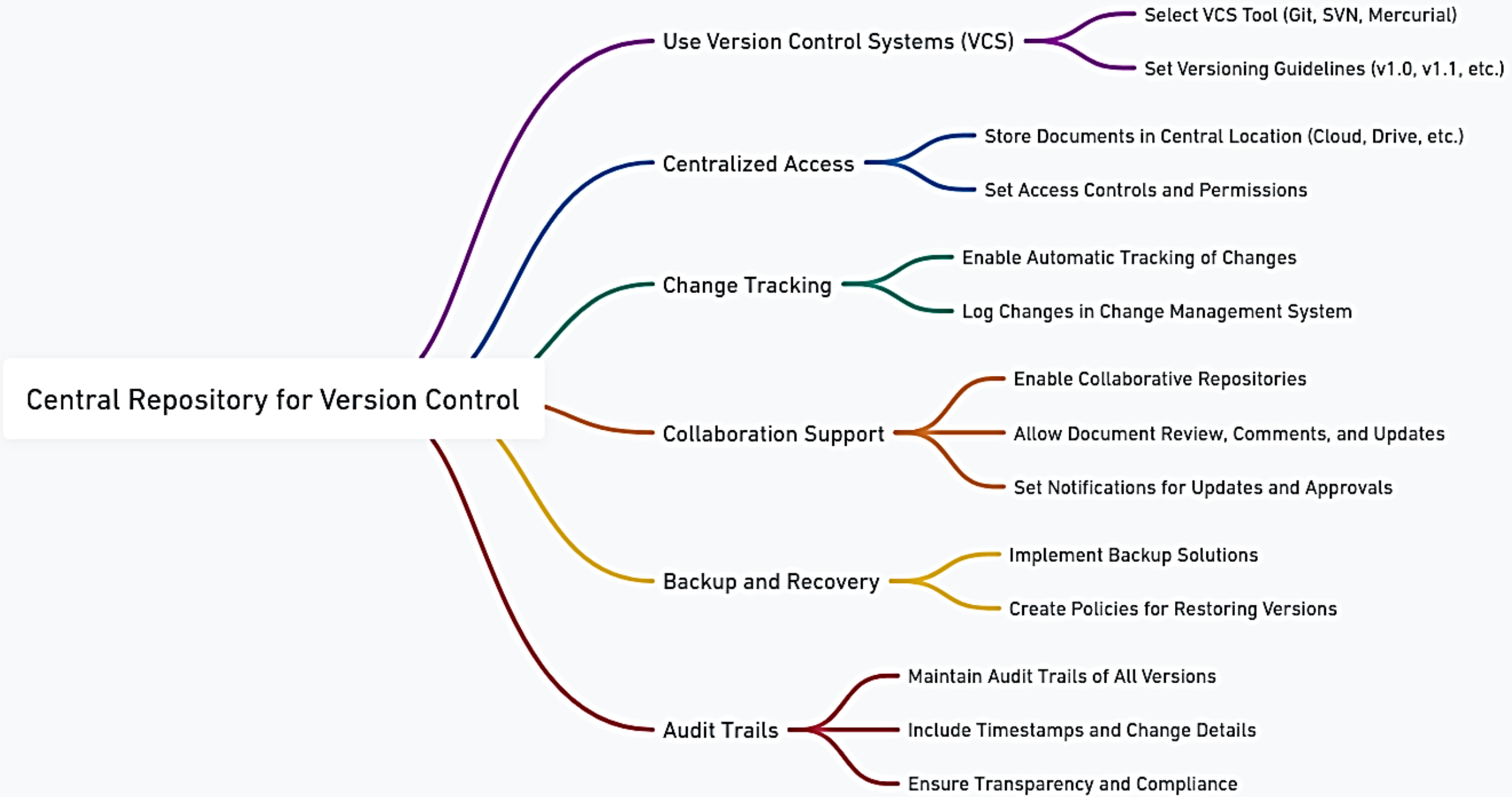

Managing Requirements
Maintaining Traceability
Link each requirement to its source
Use Requirement Traceability Matrix (RTM)
Track changes and maintain logs
Negotiating Specifications with Clients
Identify key client needs and priorities
Facilitate discussions and workshops
Document agreed-upon changes and revisions
Documenting Requirements
Use clear, unambiguous language
Follow industry standards for requirement documentation
Maintain a central repository for version control

Maintain a Central Repository for Version Control

- A central repository is critical for storing and managing requirement documents effectively.
- It ensures that all stakeholders have access to the latest version of requirements, avoids duplication, and provides a clear history of changes.
- **Use Version Control Systems (VCS)**
  - Utilize tools like Git, SVN, or Mercurial to track changes to requirement documents, maintain multiple versions, and allow rollbacks if needed.
  - Create clear versioning guidelines (e.g., v1.0 for the initial draft, v1.1 for minor edits, and v2.0 for significant updates).
- **Centralized Access**
  - Store documents in a central location, such as cloud-based platforms (e.g., Google Drive, SharePoint, or Confluence) or specialized requirement management tools like IBM DOORS, Jira, or Helix RM.
  - Set appropriate access controls and permissions to ensure data security and integrity.

Maintain a Central Repository for Version Control

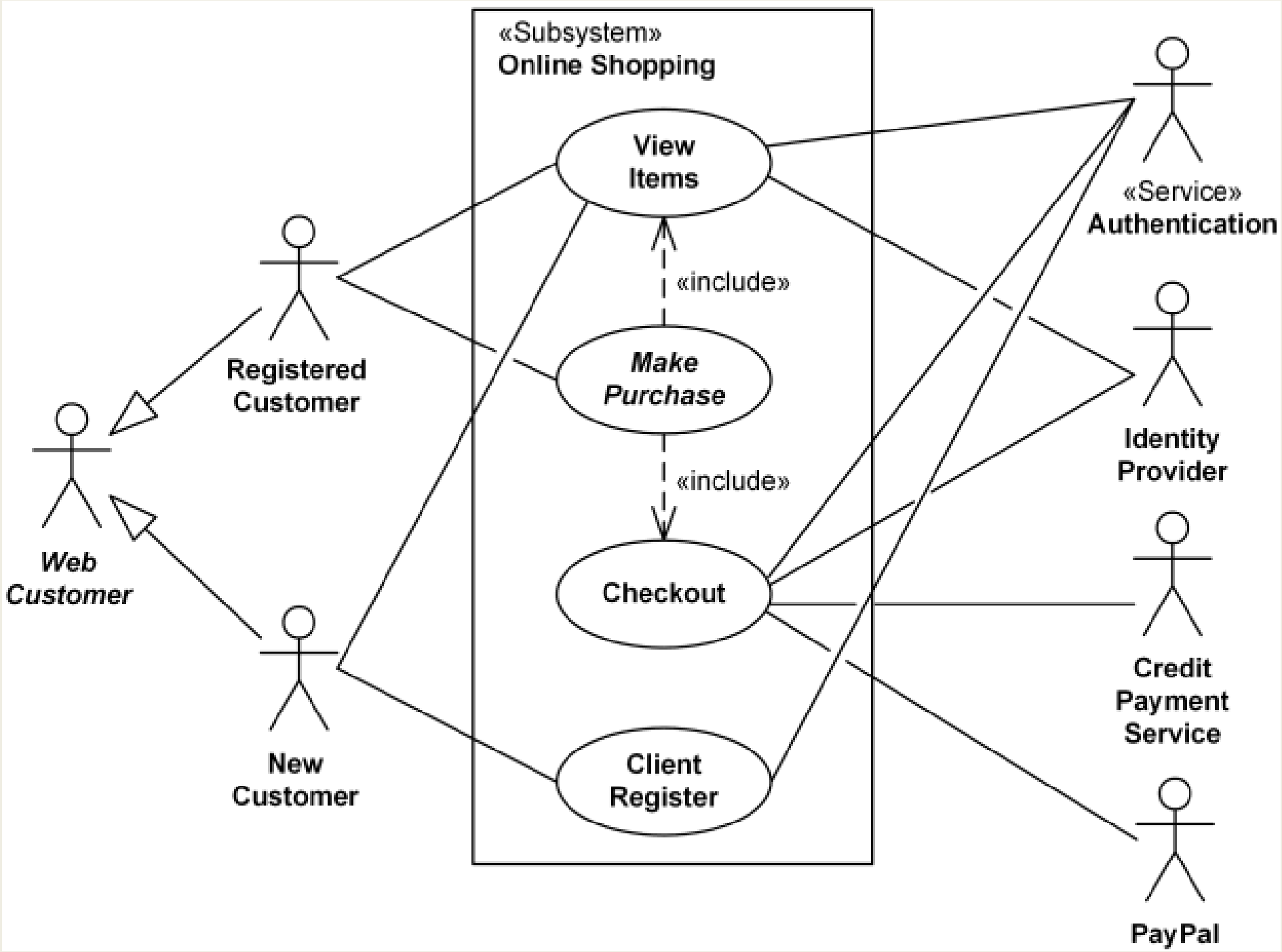
- **Change Tracking:**
  - Enable automatic tracking of changes, including who made the change, when it was made, and why.
  - Log all changes in a change management system for easy reference.
- **Collaboration Support:**
  - Use collaboration-friendly repositories that allow multiple stakeholders to review, comment on, and update documents simultaneously.
  - Incorporate communication features like notifications for updates and version approvals.
- **Backup and Recovery:**
  - Implement robust backup solutions to prevent data loss in case of accidental deletion or technical issues.
  - Establish clear policies for restoring previous versions if necessary.
- **Audit Trails:**
  - Maintain an audit trail of all document versions, including timestamps and change details, to ensure transparency and compliance with organizational and legal requirements.


Central Repository for Version Control
Use Version Control Systems (VCS)
Select VCS Tool (Git, SVN, Mercurial)
Set Versioning Guidelines (v1.0, v1.1, etc.)
Centralized Access
Store Documents in Central Location (Cloud, Drive, etc.)
Set Access Controls and Permissions
Change Tracking
Enable Automatic Tracking of Changes
Log Changes in Change Management System
Collaboration Support
Enable Collaborative Repositories
Allow Document Review, Comments, and Updates
Set Notifications for Updates and Approvals
Backup and Recovery
Implement Backup Solutions
Create Policies for Restoring Versions
Audit Trails
Maintain Audit Trails of All Versions
Include Timestamps and Change Details
Ensure Transparency and Compliance

Use Case Diagrams

- A use case diagram is a visual representation of the functional requirements of a system. It shows the relationship between the system and its actors and the interactions between them.
- **Key Components of a Use Case Diagram:**
  - *Actors: Entities that interact with the system. They can be human users, external systems, or devices.*
  - *Use Cases: The specific functions or services that the system provides.*
- **Relationships:**
  - *Association: A relationship between an actor and a use case, indicating that the actor can initiate or participate in the use case.*
  - *Include: A relationship between two use cases, indicating that one use case includes the functionality of another.*
  - *Extend: A relationship between two use cases, indicating that one use case can extend the behavior of another under certain conditions.*

Use Case Diagram



Explanation

- **Actors**: Customer, Admin
  - Use Cases: Browse Products, Add to Cart, Checkout, Manage Orders, Manage Products
- **Benefits of Using Use Case Diagrams**:
  - **Clear Communication**: Visually represents system functionality.
  - **Requirement Identification**: Helps identify and document functional requirements.
  - **System Design**: Provides a basis for designing system components and user interfaces.
  - **Test Case Design**: Can be used to derive test cases for system testing

Best Practices for Creating Use Case Diagrams

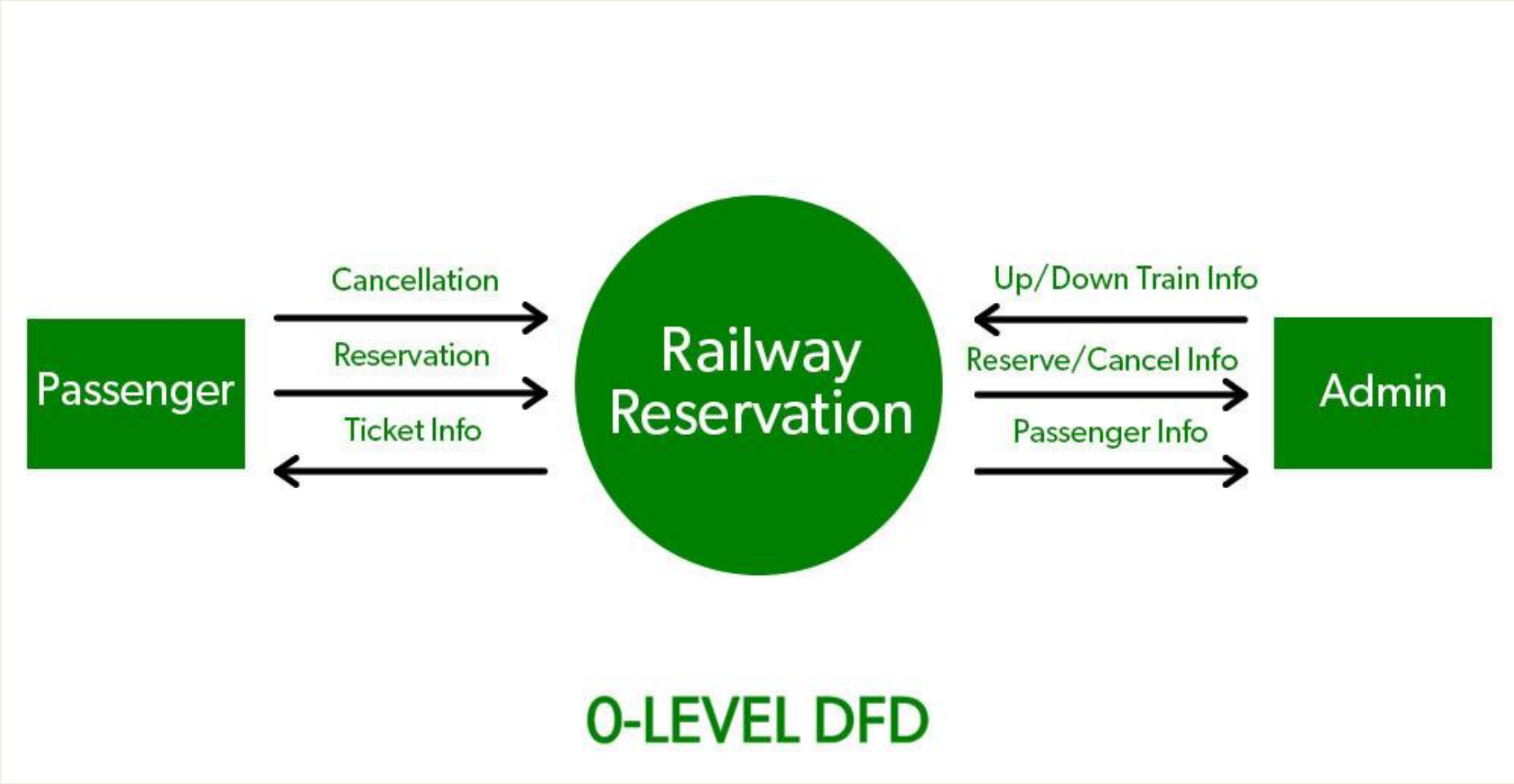
- **Identify Actors:** Determine who will interact with the system.
- **Identify Use Cases**: Identify the specific functions the system will provide.
- **Define Relationships**: Establish relationships between actors and use cases.
- **Add Details**: Include brief descriptions of each use case.
- **Review and Refine**: Iterate on the diagram to ensure accuracy and completeness.

By using use case diagrams effectively, we can create a clear and concise representation of our system's functionality, facilitating better communication and collaboration among stakeholders.

Process Models

- **Process models** are visual representations of business processes, depicting the flow of activities and decisions involved in accomplishing a specific goal.
- They are essential tools for **understanding, analyzing**, and **improving business** operations.
- Types of Process Models:
  - ***Data Flow Diagrams (DFDs):***
    - Show the flow of data through a system.
    - Focus on the data transformations and the entities involved.
    - Useful for understanding the overall data flow in a system.

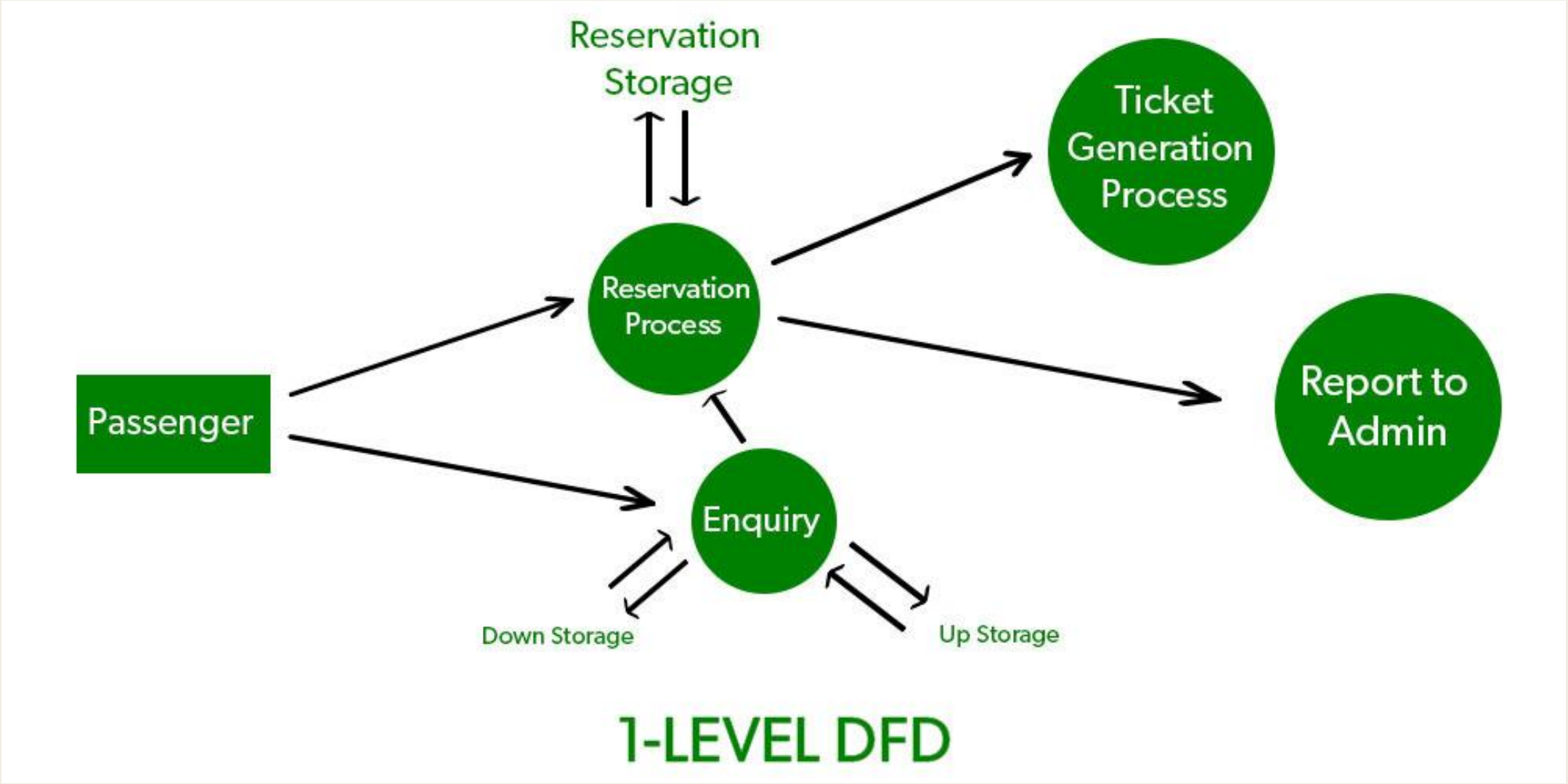
Top level DFD / Context Diagram: Level 0



Level 0

- Level 0 is the highest-level Data Flow Diagram (DFD), which provides an overview of the entire system.
- It shows the major processes, data flows, and data stores in the system, without providing any details about the internal workings of these process
- It is also known as a context diagram.
- It’s designed to be an abstraction view, showing the system as a single process with its relationship to external entities.
- It represents the entire system as a single bubble with input and output data indicated by incoming/outgoing arrows.

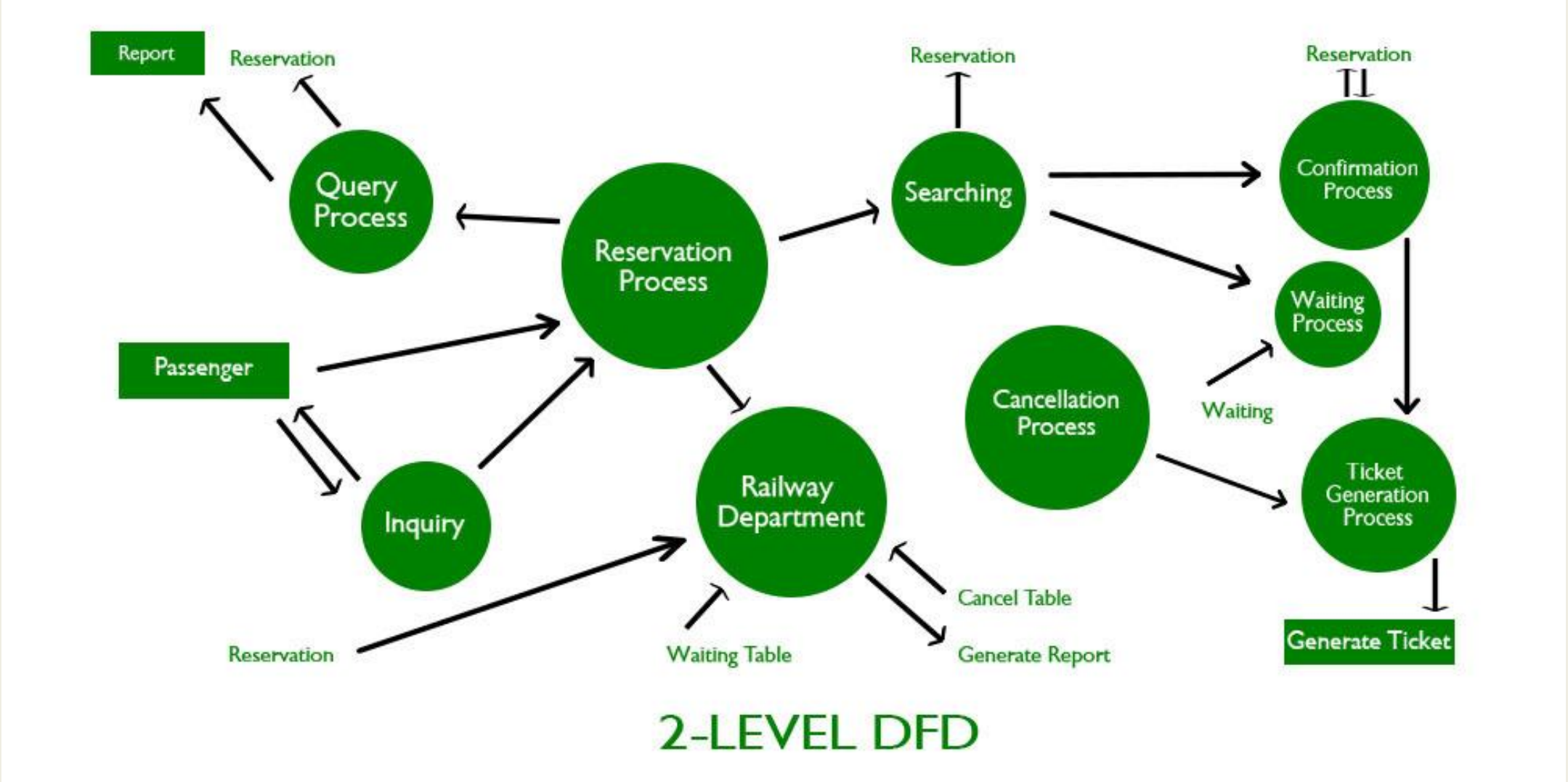
DFD: Level 1


Reservation
Storage
Ticket
Generation
Process
Reservation
Process
Passenger
Report to
Admin
Enquiry
Down Storage
Up Storage
1-LEVEL DFD

DFD: 1-Level

- 1-Level provides a more detailed view of the system by breaking down the major processes identified in the level 0 Data Flow Diagram (DFD) into sub-processes.
- Each sub-process is depicted as a separate process on the level 1 Data Flow Diagram (DFD).
- The data flows and data stores associated with each sub-process are also shown.
- In 1-level Data Flow Diagram (DFD), the context diagram is decomposed into multiple bubbles/processes.
- In this level, we highlight the main functions of the system and breakdown the high-level process of 0-level Data Flow Diagram (DFD) into subprocesses.

DFD: Level 2


Report
Reservation
Reservation
Reservation
Query Process
Reservation Process
Searching
Confirmation Process
Waiting Process
Passenger
Cancellation Process
Waiting
Inquiry
Railway Department
Ticket Generation Process
Cancel Table
Reservation
Waiting Table
Generate Report
Generate Ticket
2-LEVEL DFD

Advantages of using Data Flow Diagrams (DFD)

- **Easy to understand**: DFDs are graphical representations that are easy to understand and communicate, making them useful for non-technical stakeholders and team members.
- **Improves system analysis**: DFDs are useful for analyzing a system's processes and data flow, which can help identify inefficiencies, redundancies, and other problems that may exist in the system.
- **Supports system design**: DFDs can be used to design a system's architecture and structure, which can help ensure that the system is designed to meet the requirements of the stakeholders.
- **Enables testing and verification**: DFDs can be used to identify the inputs and outputs of a system, which can help in the testing and verification of the system's functionality.
- **Facilitates documentation**: DFDs provide a visual representation of a system, making it easier to document and maintain the system over time.

Disadvantages of using Data Flow Diagram (DFD)

1. **Can be time-consuming:** Creating DFDs can be a time-consuming process, especially for complex systems.
2. **Limited focus:** DFDs focus primarily on the flow of data in a system, and may not capture other important aspects of the system, such as user interface design, system security, or system performance.
3. **Can be difficult to keep up-to-date:** DFDs may become out-of-date over time as the system evolves and changes.
4. **Requires technical expertise:** While DFDs are easy to understand, creating them requires a certain level of technical expertise and familiarity with the system being analyzed.

Thank you!